САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бекэнд-разработка

Отчёт

Домашнее задание №5 «Реализация межсервисного взаимодействия посредством очередей сообщений»

Выполнил:

Клименко Богдан Владимирович

Группа БР 1.1

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить брокер сообщений RabbitMQ для реализации асинхронного межсервисного взаимодействия в микросервисной архитектуре, созданной в рамках ЛР2. Обеспечить публикацию событий одними сервисами и их обработку другими для достижения слабой связанности (loose coupling) и отказоустойчивости.

Ход работы

1. Настройка RabbitMQ
Брокер сообщений RabbitMQ развёрнут в Docker-контейнере с образом rabbitmq:3-management:

docker run -d --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:3-management

- Порт 5672 используется для AMQP-подключений из сервисов.
- Порт 15672 — для веб-панели управления (http://localhost:15672, логин/пароль guest/guest).

2. Архитектура взаимодействия
Взаимодействие построено по принципу **«издатель-подписчик» (publisher-subscriber)** с использованием **topic-обменников** (exchanges). Это позволяет гибко маршрутизировать события по ключам (routing keys).

Роли сервисов:
- **Post Service (издатель):** публикует события, связанные с постами.
- **Comment Service (издатель):** публикует события, связанные с комментариями.
- **Social Service (подписчик):** подписан на события от Post и Comment Service и реагирует на них, создавая или удаляя уведомления.
-

Схема потоков событий:
- **Обменник** post.events (тип topic):
  - like.created: { userId, postId, postOwnerId } — пользователь лайкнул пост.
  - like.removed: { userId, postId } — пользователь убрал лайк.
  - post.deleted: { postId, userId } — пост удалён.
- **Обменник** comment.events (тип topic):
  - comment.created: { commentId, userId, postId, postOwnerId, hasPhoto } — создан комментарий.

3. Реализация издателей (Post Service, Comment Service)
В каждом сервисе создан модуль utils/rabbitmq.ts, который:

1. Устанавливает соединение с RabbitMQ.
2. Создаёт канал.
3. Объявляет обменник с durable: true (чтобы он выжил после перезапуска брокера).
4. Предоставляет функцию publishEvent(routingKey, payload) для отправки сообщений.

Пример функции публикации:

```
export async function publishEvent(routingKey: string, payload: object):
Promise<void> {
  if (!channel) {
    console.warn("[post-service] RabbitMQ channel not ready, skipping event:",
routingKey);
    return;
  }
  try {
    const message = Buffer.from(JSON.stringify(payload));
    channel.publish(EXCHANGE, routingKey, message, { persistent: true });
    console.log(`[post-service] Event published: ${routingKey}`, payload);
  } catch (error) {
    console.error("[post-service] publishEvent error:", error);
  }
}
```

Публикация событий происходит в соответствующих контроллерах.
Например, в PostController.likePost:

```
      if (post.userId !== userId) {
        await publishEvent("like.created", {
          userId,
          postId,
          postOwnerId: post.userId,
        });
      }
```

4. Реализация подписчика (Social Service)
Social Service при запуске выполняет следующие действия:

1. Устанавливает соединение с RabbitMQ.
2. Создаёт канал.
3. Объявляет те же обменники (post.events, comment.events), чтобы гарантировать их существование.
4. Создаёт долговечную очередь social-service-events (durable: true).
5. Привязывает (bind) очередь к обменникам по нужным ключам маршрутизации:

- like.created
- like.removed
- comment.created
- post.deleted

6. Запускает потребителя (consume), который обрабатывает входящие сообщения.

Пример обработки события like.created:

```
async function handleLikeCreated(payload: {
  userId: number;
  postId: number;
  postOwnerId: number;
}) {
  try {
    if (payload.userId === payload.postOwnerId) return;
    const notifRepo = AppDataSource.getRepository(Notification);
    const notif = notifRepo.create({
      userId: payload.postOwnerId,
      type: "like",
      relatedUserId: payload.userId,
      relatedPostId: payload.postId,
    });
    await notifRepo.save(notif);
    console.log("[social-service] Notification created for like:", notif.id);
  } catch (error) {
    console.error("[social-service] handleLikeCreated error:", error);
  }
}
```

5. Результаты тестирования

Работа межсервисного взаимодействия была подтверждена логами всех сервисов при выполнении сквозного сценария (лайк и комментарий от второго пользователя, удаление поста).

Post Service (издатель):
[post-service] Event published: like.created { userId: 2, postId: 1, postOwnerId: 1 }
[post-service] Event published: post.deleted { postId: 1, userId: 1 }

Comment Service (издатель):
[comment-service] Event published: comment.created { commentId: 4, userId: 2, postId: 2, postOwnerId: 1, hasPhoto: false }

Social Service (подписчик):
[social-service] Received event: like.created { userId: 2, postId: 1, postOwnerId: 1 }

[social-service] Notification created for like: 1
[social-service] Received event: post.deleted { postId: 1, userId: 1 }
[social-service] Notifications removed for post: 1

Вывод

В ходе выполнения домашнего задания №5 было реализовано асинхронное межсервисное взаимодействие с использованием брокера сообщений RabbitMQ. Публикация событий в Post и Comment Service и их обработка в Social Service позволили добиться **слабой связанности** сервисов: отправители не знают о получателях и не зависят от их доступности. Использование durable-очередей и persistent-сообщений обеспечивает **надёжность** доставки даже при перезапуске брокера. Данный подход полностью соответствует требованиям задания и является основой для построения отказоустойчивых распределённых систем.